1ST EDITION

React Anti-Patterns

Build efficient and maintainable React applications with test-driven development and refactoring

JUNTAO QIU

React Anti-Patterns

Build efficient and maintainable React applications with test-driven development and refactoring

Juntao Qiu

BIRMINGHAM—MUMBAI

React Anti-Patterns

Group Product Manager: Rohit Rajkumar
Publishing Product Manager: Jane D'Souza
Senior Editor: Hayden Edwards
Technical Editor: K Bimala Singha
Copy Editor: Safis Editing
Project Coordinator: Aishwarya Mohan
Proofreader: Safis Editing
Indexer: Subalakshmi Govindhan
Production Designer: Shankar Kalbhor
Marketing Coordinators: Namita Velgekar and Nivedita Pandey

First published: January 2024

Production reference: 1071223

Published by
Packt Publishing Ltd.
Grosvenor House
11 St Paul's Square
Birmingham
B3 1RB, UK

ISBN 978-1-80512-397-2

www.packtpub.com

To my loving wife, Mansi, and joyful daughter, Luna – your laughter was my daily respite, and your support allowed me the time at my desk to bring this book to fruition. While the pages reflect my work, they also echo your sacrifice and love. Thank you.

– Juntao

Contributors

About the author

Juntao Qiu is an accomplished software developer with over 15 years of industry experience, dedicated to helping others write better code. With a strong passion for crafting maintainable and high-quality code, he has become a trusted resource in the industry. As an author, Juntao has shared his expertise through influential books such as *Maintainable React* (2022) and *Test-Driven Development with React and TypeScript – Second Edition* (2023).

He orchestrates a blend of code snippets, illustrations, and before and after comparisons to unravel intricate concepts, making them accessible to fellow developers. His insightful blog posts at `https://juntao.substack.com/` and educational videos on YouTube (`https://www.youtube.com/@icodeit.juntao`) are a testament to his commitment to sharing knowledge and fostering a deeper understanding of coding practices within the developer community.

I am immensely thankful to my Atlassian teammates, Alex Reardon, Daniel Del Core, Michael Dougall, and many others. Many of the design patterns elucidated in this book germinated from our collective intellect while engrossed in Atlassian Design System components. A special shoutout to Tom Gasson, with whom I had an enriching time developing an internal system; our dialogues, particularly on React Context, have been significantly enlightening.

I also extend my appreciation to James Sinclair and Jason Sheehy for our enlightening discussions on TDD within our current monorepo. A special thank you goes to Vanessa Goah and Mirela Tomicic, who were instrumental during my recent onboarding. Their willingness to engage with my inquiries and provide clarity amid the myriad documents and conventions significantly accelerated my adaptation to the new project and enriched my understanding of the patterns embedded within the code base – a treasure trove of ideas discussed in this book.

My heartfelt appreciation goes to my former colleagues at Thoughtworks. Martin Fowler, your guidance has been a beacon, refining my focus and teaching me the essence of audience-centric writing. I am grateful to Xiaojun Ren for the stimulating design discussions, which were often accompanied by our serene strolls through the chilly suburbs of Melbourne. Thanks to Andy Marks and Cam Jackson for their meticulous reviews and insightful suggestions on technical nuances and linguistic improvements. The consultancy nature of the work at Thoughtworks provided a fertile ground to cross-pollinate ideas with various teams and individuals, an experience I hold dear.

A big thank you to my editor, Hayden Edwards, whose professional editing suggestions were instrumental in enhancing the narrative flow of this book. The journey from a manuscript to a published book is indeed a collaborative venture, and the unseen yet pivotal contributions from the team behind the scenes have been invaluable. Your expertise and dedication have breathed life into this work, and for that, I am profoundly grateful.

About the reviewers

Dennis Persson is a Swedish full stack web and mobile developer with over 10 years of expertise and a master of science degree in computer science and engineering. His academic journey includes a noteworthy tenure at Linköping University, where he imparted knowledge in more than a dozen courses, including both theoretical and practical applications of cutting-edge technologies.

Parallel to his current role as an application developer, Dennis manages a tech blog, providing insights into the evolving tech landscape. This unique combination of academic depth, practical expertise, and knowledge-sharing positions Dennis as a driving force at the intersection of education and industry innovation.

Krishnan Raghavan is an IT professional with over 20 years of experience in the areas of software development and delivery excellence across multiple domains and technologies, ranging from C++, Java, and Python to Angular, Golang, and data warehousing.

When not working, Krishnan likes to spend time with his wife and daughter, reading fiction and nonfiction as well as technical books, and participating in hackathons. Krishnan tries to give back to the community by being part of the GDG Pune volunteer group, helping the team to organize events.

Table of Contents

Preface xiii

Part 1: Introducing the Fundamentals

1

Introducing React Anti-Patterns 3

Technical requirements 4
Understanding the difficulty of building UIs 4
Understanding the state management 6
Exploring "unhappy paths" 10
Errors thrown from other components 10
Learning the unexpected user behavior 11
Exploring common anti-patterns in React 13
Props drilling 13
In-component data transformation 14
Complicated logic in views 15
Lack of tests 16
Duplicated code 18
Long component with too much responsibility 18
Unveiling our approach to demolishing anti-patterns 19
Summary 20

2

Understanding React Essentials 21

Technical requirements 22
Understanding static components in React 22
Creating components with props 22
Breaking down UIs into components 24
Managing internal state in React 27
Understanding the rendering process 28
Exploring common React Hooks 29
useState 30
useEffect 31
useCallback 35
The React Context API 37
Summary 41

3

Organizing Your React Application 43

Technical requirements 44
Understanding the problem of a less-structured project 44
Understanding the complications of frontend applications 45
Exploring common structures in React applications 47
Feature-based structure 47
Component-based structure 49
Atomic design structure 50
The MVVM structure 52
Keeping your project structure organized 54
Implementing the initial structure 54
Adding an extra layer to remove duplicates 55
Naming files 56
Exploring a more customized structure 58
Summary 59

4

Designing Your React Components 61

Technical requirements 62
Exploring the single responsibility principle 62
Don't repeat yourself 65
Using composition 69
Combining component design principles 72
Summary 80

Part 2: Embracing Testing Techniques

5

Testing in React 83

Technical requirements 83
Understanding why we need tests 84
Learning about different types of tests 84
Testing individual units with Jest 87
Writing your first test 87
Grouping tests 88
Testing React components 89
Learning about integration tests 91
Learning about E2E tests using Cypress 95
Installing Cypress 96
Running our first E2E test 98
Intercepting the network request 100
Summary 102

6

Exploring Common Refactoring Techniques 103

Technical requirements 104
Understanding refactoring 106
The common mistakes of refactoring 107
Adding tests before refactoring 108
Using Rename Variable 110
Using Extract Variable 111
Using Replace Loop with Pipeline 112
Using Extract Function 113
Using Introduce Parameter Object 114
Using Decompose Conditional 115
Using Move Function 116
Summary 118

7

Introducing Test-Driven Development with React 119

Technical requirements 120
Understanding TDD 120
Different styles of TDD 122
Focusing on user value 123
Introducing tasking 124
Introducing the online pizza store application 125
Breaking down the application requirements 126
Implementing the application headline 130
Implementing the menu list 132
Creating the shopping cart 135
Adding items to the shopping cart 137
Refactoring the code 140
Summary 144

Part 3: Unveiling Business Logic and Design Patterns

8

Exploring Data Management in React 147

Technical requirements 148
Understanding business logic leaks 148
Introducing the ACL 150
Introducing a typical usage 151
Using the fallback or default value 153
Exploring the prop drilling issue 155
Using the Context API to resolve prop drilling 161
Summary 164

9

Applying Design Principles in React 165

Technical requirements 166
Revisiting the Single Responsibility Principle 166
Exploring the render props pattern 167
Using composition to apply the SRP 169
Embracing the Dependency Inversion Principle 171
Understanding how the DIP works 172
Applying the DIP in an analytics button 174
Understanding Command and Query Responsibility Segregation in React 178
Introducing useReducer 180
Using a reducer function in a context 182
Summary 185

10

Diving Deep into Composition Patterns 187

Technical requirements 188
Understanding composition through higher-order components 188
Reviewing higher-order functions 188
Introducing HOCs 189
Implementing an ExpandablePanel component 190
Exploring React Hooks 195
Unveiling remote data fetching 198
Refactoring for elegance and reusability 199
Developing a drop-down list component 200
Implementing keyboard navigation 205
Maintaining simplicity in the Dropdown component 208
Introducing the Headless Component pattern 210
The advantages and drawbacks of Headless Component pattern 211
Libraries and further learnings 212
Summary 213

Part 4: Engaging in Practical Implementation

11

Introducing Layered Architecture in React 217

Technical requirements 218
Understanding the evolution of a React application 218
Single-component applications 218
Multiple-component applications 219
State management with Hooks 220

Extracting business models 221
Layered frontend application 221

Enhancing the Code Oven application 223

Refactoring the MenuList through a custom Hook 227
Transitioning to a class-based model 229

Implementing the ShoppingCart component 231

Applying discounts to Items 233
Exploring the Strategy pattern 238

Delving into layered architecture 242

The layered structure of the application 243
Advantages of layered architecture 245

Summary 245

12

Implementing an End-To-End Project 247

Technical requirements 248

Getting the OpenWeatherMap API key 248
Preparing the project's code base 248

Reviewing the requirements for the weather application 249

Crafting our initial acceptance test 250

Implementing a City Search feature 251

Introducing the OpenWeatherMap API 252
Stubbing the search results 253
Enhancing the search result list 256

Implementing an ACL 259

Implementing an Add to Favorite feature 262

Modeling the weather 267
Refactoring the current implementation 270
Enabling multiple cities in the favorite list 276
Refactoring the weather list 279

Fetching previous weather data when the application relaunches 280

Summary 284

13

Recapping Anti-Pattern Principles 285

Revisiting common anti-patterns 286

Props drilling 286
Long props list/big component 286
Business leakage 287
Complicated logic in views 287
Lack of tests (at each level) 287
Code duplications 287

Skimming through design patterns 288

Higher-order components 288
Render props 288
Headless components 288
Data modeling 288
Layered architecture 288
Context as an interface 289

Revisiting foundational design principles 290
Single Responsibility Principle 290
Dependency Inversion Principle 290
Don't Repeat Yourself 290
Anti-Corruption Layers 290
Using composition 291

Recapping techniques and practices 291
Writing user acceptance tests 291
Test-Driven Development 292
Refactoring and common code smells 292

Additional resources 293

Summary 294

Index 295

Other Book You May Enjoy 302

Preface

Building frontend applications is challenging, especially when constructing large ones, and the difficulty escalates without proper guidance. Unfortunately, many React-based applications fall into this scenario due to the library's UI-centric nature, leaving developers to navigate the other complexities of frontend development on their own. There are numerous other considerations such as asynchronous network requests, accessibility, performance, and state management, to name a few. These factors contribute to the complexity of frontend applications. As the scale of the application grows, maintaining the code becomes an arduous task. Adding new features requires considerably more time than it first appears, and identifying defects (and then fixing them) is equally challenging, if not more so.

However, these challenges are surmountable. We can learn to identify common anti-patterns that cause problems, then employ established patterns, design principles, and practices to address and rectify these issues. History teaches us that solutions derived in one field often find relevance in others, especially when it comes to fundamental design principles such as the Single Responsibility Principle, the Dependency Inversion Principle, and Don't Repeat Yourself. These principles guided the construction of UNIX systems back in the 1970s and Java Swing applications in the 1990s, and they remain valid today. They will undoubtedly continue to be pertinent for future frameworks and libraries.

This book seeks to delve into these problems and examine how established patterns and practices can mitigate the challenges of building large applications. We'll see how design principles and design patterns can simplify the design, making the code easier to understand, modify, and maintain in the long run. Through this exploration, readers will gain a deeper understanding of how to navigate the multifaceted world of frontend development with React, ensuring their applications are both robust and maintainable.

Who this book is for

This book is for React developers who are interested in improving the maintainability and efficiency of their code. Whether you're just starting out or have some experience under your belt, there's something here for you. It's beneficial to have a basic understanding of React, but the book aims to guide you through the concepts in a straightforward manner.

The focus is on identifying common anti-patterns and addressing them with established design principles and patterns. Through practical examples and a step-by-step approach, you'll learn how to simplify your code for better understanding, easier modifications, and long-term maintenance.

What this book covers

In *Chapter 1*, *Introducing React Anti-Patterns*, you'll get a closer look at the hurdles of building user interfaces, handling state management, addressing "unhappy paths," and identifying common anti-patterns in React.

In *Chapter 2, Understanding React Essentials*, you will delve into the basics of React covering static components, props, UI breakdown, state management, the rendering process, and common React Hooks to lay a solid foundation for subsequent chapters.

In *Chapter 3, Organizing Your React Application*, you will learn about different types of project structures in React, exploring their advantages, drawbacks, and practical applications.

In *Chapter 4, Designing your React Components*, you will learn to identify common anti-patterns in React component design and explore fundamental design principles including the Single Responsibility Principle and Don't Repeat Yourself to improve component structure.

In *Chapter 5, Testing in React*, you will learn about the significance of software testing, explore various types of tests such as unit, integration, and end-to-end testing, and get acquainted with popular testing tools including Cypress and Jest, setting a strong foundation for complex testing scenarios in React applications.

In *Chapter 6, Exploring Common Refactoring Techniques*, you will learn about the essence of refactoring and delve into various refactoring techniques, such as Rename Variable, Extract Variable, and Replace Loop with Pipeline, to enhance code maintainability and readability.

In *Chapter 7, Introducing Test-Driven Development with React*, you'll learn the core principles of test-driven development through a practical example, while building various features of a pizza store's menu page in a React application.

In *Chapter 8, Exploring Data Management in React*, you'll delve into the common challenges of state management in React, such as business logic leaks and prop drilling, and explore solutions including employing an Anti-Corruption Layer and utilizing the React context API to enhance code maintainability and user experience.

In *Chapter 9, Applying Design Principles in React*, you'll revisit the Single Responsibility Principle, embrace the Dependency Inversion Principle, and understand the application of Command and Query Responsibility Segregation in React to fortify your knowledge of key design principles to aid you in mastering React.

In *Chapter 10, Diving Deep into Composition Patterns*, you'll delve into composition through higher-order components and custom Hooks, and explore the headless component pattern. You'll gain an appreciation of composition techniques for creating scalable, maintainable, and user-friendly UIs in React.

In *Chapter 11, Introducing Layered Architecture in React*, you'll explore Layered Architecture, delve into Application Concern Layers, define data models, and learn strategy patterns through a practical example, understanding their significance for large-scale applications.

In *Chapter 12, Implementing an End-To-End Project*, you'll traverse the complete process of developing a weather application, from understanding requirements to implementing features such as City Search and Add To Favourite, while ensuring the code remains maintainable, understandable, and extensible.

In *Chapter 13, Recapping Anti-Pattern Principles*, we'll take a concise look back at common anti-patterns, React design patterns, and fundamental principles, and recap the techniques and practices discussed earlier in the book, providing a succinct refresher before you continue applying these insights to your own projects.

To get the most out of this book

To delve deeply into this book, having a text editor at hand is crucial; choices such as Visual Studio Code or Vim are commendable, but any other editor of your preference will serve well. An alternative is a full-featured **Integrated Development Environment** (**IDE**) such as WebStorm or IntelliJ, which, while not mandatory, could significantly ramp up your efficiency.

A command-line interface is another requisite; for Mac or Linux users, the setup is already in place, but Windows users might need to make an installation—Windows Terminal is a good choice, providing a modern terminal and command-line experience. Preparing these tools in advance will ensure a seamless journey as you traverse through the content of this book.

Software/hardware covered in the book	**Operating system requirements**
React 16+	Windows, macOS, or Linux
TypeScript 4.9.5	
Visual Studio Code or WebStorm	
Terminal/Window Terminal (for Windows users)	

Download the example code files

You can download the example code files for this book from GitHub at `https://github.com/PacktPublishing/React-Anti-Patterns/`. If there's an update to the code, it will be updated in the GitHub repository.

We also have other code bundles from our rich catalog of books and videos available at `https://github.com/PacktPublishing/`. Check them out!

Conventions used

There are a number of text conventions used throughout this book.

`Code in text`: Indicates code words in text, database table names, folder names, filenames, file extensions, pathnames, dummy URLs, user input, and Twitter handles. Here is an example: "We can then pass the desired `heading` and `summary` to the `Article` component."

A block of code is set as follows:

```
<Article
  heading="Think in components"
  summary="It's important to change your mindset when coding with
React."
/>
```

When we wish to draw your attention to a particular part of a code block, the relevant lines or items are set in bold:

```
<article>
  <h3>Think in components</h3>
  <p>It's important to change your mindset when coding with React.</p>
</article>
```

Any command-line input or output is written as follows:

```
$ mkdir css
$ cd css
```

Bold: Indicates a new term, an important word, or words that you see onscreen. For instance, words in menus or dialog boxes appear in **bold**. Here is an example: "Select **System info** from the **Administration** panel."

> **Tips or important notes**
>
> Appear like this.

Get in touch

Feedback from our readers is always welcome.

General feedback: If you have questions about any aspect of this book, email us at `customercare@packtpub.com` and mention the book title in the subject of your message.

Errata: Although we have taken every care to ensure the accuracy of our content, mistakes do happen. If you have found a mistake in this book, we would be grateful if you would report this to us. Please visit `www.packtpub.com/support/errata` and fill in the form.

Piracy: If you come across any illegal copies of our works in any form on the internet, we would be grateful if you would provide us with the location address or website name. Please contact us at `copyright@packt.com` with a link to the material.

If you are interested in becoming an author: If there is a topic that you have expertise in and you are interested in either writing or contributing to a book, please visit `authors.packtpub.com`.

Share Your Thoughts

Once you've read *React Anti-Patterns*, we'd love to hear your thoughts! Please click here to go straight to the Amazon review page for this book and share your feedback.

Your review is important to us and the tech community and will help us make sure we're delivering excellent quality content.

Download a free PDF copy of this book

Thanks for purchasing this book!

Do you like to read on the go but are unable to carry your print books everywhere?

Is your eBook purchase not compatible with the device of your choice?

Don't worry, now with every Packt book you get a DRM-free PDF version of that book at no cost.

Read anywhere, any place, on any device. Search, copy, and paste code from your favorite technical books directly into your application.

The perks don't stop there, you can get exclusive access to discounts, newsletters, and great free content in your inbox daily

Follow these simple steps to get the benefits:

1. Scan the QR code or visit the link below

`https://packt.link/free-ebook/9781805123972`

2. Submit your proof of purchase
3. That's it! We'll send your free PDF and other benefits to your email directly

Part 1: Introducing the Fundamentals

In the first part of this book, you will set foot into the realm of React by exploring its core essentials and understanding how to efficiently structure your application. This part will aid in building a strong foundation, crucial to navigate through the more complex aspects of React that are discussed in the following parts.

This part contains the following chapters:

- *Chapter 1*, *Introducing React Anti-Patterns*
- *Chapter 2*, *Understanding React Essentials*
- *Chapter 3*, *Organizing Your React Application*
- *Chapter 4*, *Designing Your React Components*

1

Introducing React Anti-Patterns

This book dives deep into the realm of React anti-patterns. An anti-pattern is not necessarily a technical error – the code often functions properly at first – but although it may initially seem correct, as the code base expands, these anti-patterns can become problematic.

As we navigate through the book, we'll scrutinize code samples that might not embody best practices; some could be intricate to decipher, and others, tough to modify or extend. While certain pieces of code may suffice for smaller tasks, they falter when scaled up. Moreover, we'll venture into time-tested patterns and principles from the expansive software world, seamlessly weaving them into our frontend discourse.

I aim for practicality. The code illustrations originate either from past projects or commonplace domains such as a shopping cart and a user profile component, minimizing your need to decipher domain jargon. For a holistic view, the concluding chapters showcase detailed, end-to-end examples, furnishing a more organized and immersive experience.

Specifically, in this introductory chapter, we'll address the intricacies of constructing advanced React applications, highlighting how state management and asynchronous operations can obfuscate code clarity. We'll enumerate prevalent anti-patterns and offer a glimpse into the remedial strategies detailed later in the book.

In this chapter, we will cover the following topics:

- Understanding the difficulty of building UIs
- Understanding the state management
- Exploring "unhappy paths"
- Exploring common anti-patterns in React

Technical requirements

A GitHub repository has been created to host all the code we discuss in the book. For this chapter, you can find the code at `https://github.com/PacktPublishing/React-Anti-Patterns/tree/main/code/src/ch1`.

Understanding the difficulty of building UIs

Unless you're building a straightforward, document-like web page — for example, a basic article without advanced UI elements such as search boxes or modals — the built-in languages offered by web browsers are generally insufficient. *Figure 1.1* shows an example of a website using **HTML** (**HyperText Markup Language**):

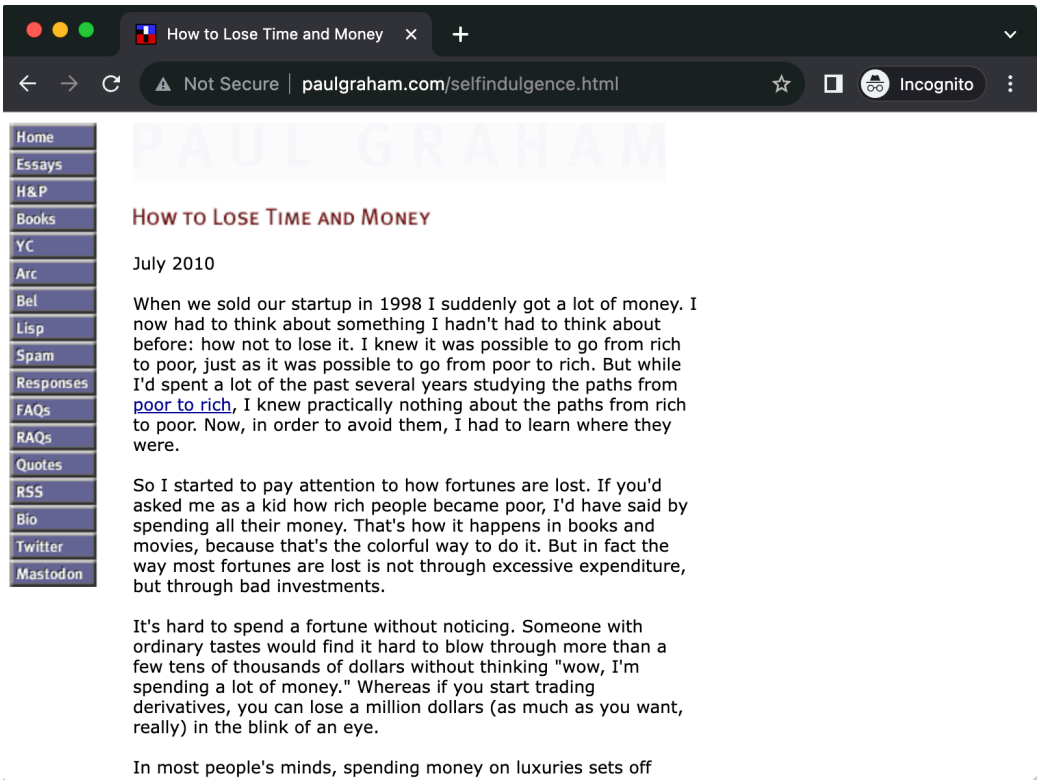

Figure 1.1: A simple HTML document website

However nowadays, most applications are more complicated and contain more elements than what this language was originally designed for.

The disparity between the language of the web and the UI experiences that people encounter daily is substantial. Whether it's a ticket booking platform, a project management tool, or an image gallery,

modern web UIs are intricate and native web languages don't readily support them. You can go the extra mile to "simulate" UI components such as accordions, toggle switches, or interactive cards, but fundamentally, you're still working with what amounts to a document, not a genuine UI component.

In an ideal world, building a UI would resemble working with a visual UI designer. Tools such as C++ Builder or Delphi, or more modern alternatives such as Figma, let you drag and drop components onto a canvas that then renders seamlessly on any screen. This isn't the case with web development. For instance, to create a custom search input, you'll need to wrap it in additional elements, fine-tune colors, adjust padding and fonts, and perhaps add an icon for user guidance. Creating an auto-suggestion list that appears right under the search box, matching its width exactly, is often far more labor-intensive than one might initially think.

As shown in *Figure 1.2*, a web page can be super complicated and look nothing like a document on the surface, although the building blocks of the page are still pure HTML:

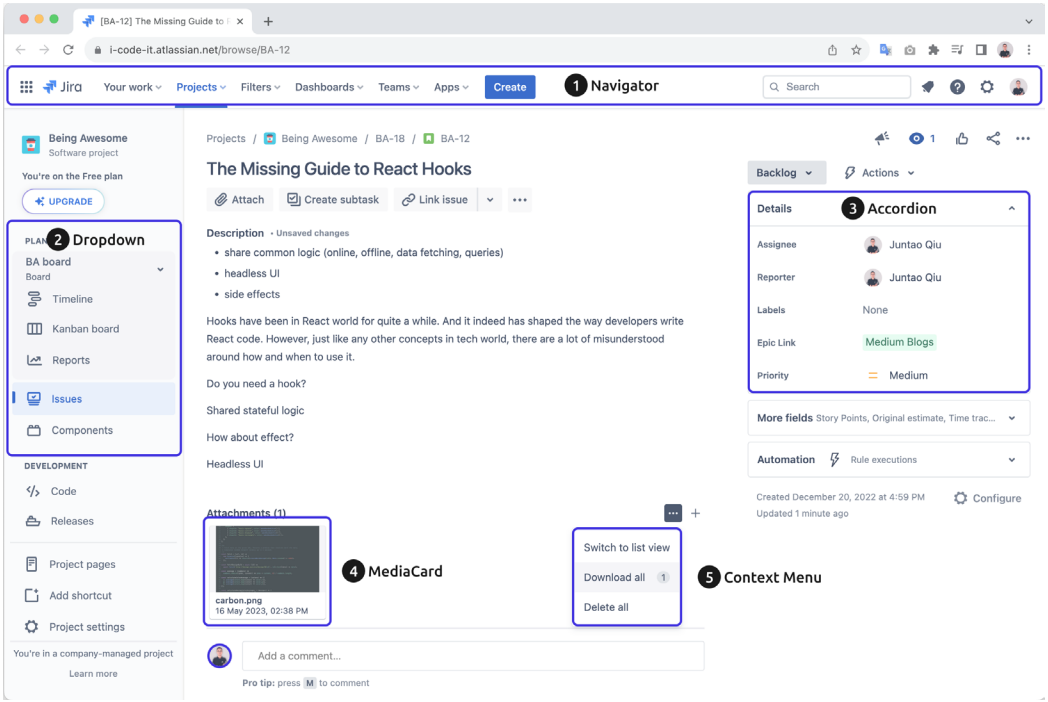

Figure 1.2: Jira issue view

This screenshot shows the issue view of Jira, a popular web-based project management tool used to track, prioritize, and coordinate tasks and projects. An issue view contains many details such as the issue's title, description, attachments, comments, and linked issues. It also contains many elements a user can interact with, such as an **Assign to me** button, the ability to change the priority of the issue, add a comment, and so on.

For such a UI, you might expect there to be a navigator component, a drop-down list, an accordion, and so on. And seemingly, they are there, as labeled in *Figure 1.2*. But they are not actually components. Instead, developers have worked hard to *simulate* these with HTML, CSS, and JavaScript.

Now that we've glanced over the language mismatch issue in web UI development, it might be helpful to delve into what's under the surface – the different states we need to manage in frontend applications. This will provide a taste of the challenges that lie ahead and shed light on why introducing patterns is a key step toward addressing them.

Understanding the state management

Managing the state in modern frontend development is a complex task. Nearly every application has to retrieve data from a remote server via a network – we can call this data **remote states**. Remote state originates from an external source, typically a backend server or API. This is in contrast to local state, which is generated and managed entirely within the frontend application itself.

There are many dark sides of remote states, making the frontend development difficult if you don't pay close attention to them. Here, I'll just list a few obvious considerations:

- *Asynchronous nature*: Fetching data from a remote source is usually an asynchronous operation. This adds complexity in terms of timing, especially when you have to synchronize multiple pieces of remote data.
- *Error handling*: Connections to remote sources might fail or the server might return errors. Properly managing these scenarios for a smooth user experience can be challenging.
- *Loading states*: While waiting for data to arrive from a remote source, the application needs to handle "loading" states effectively. This usually involves showing loading indicators or fallback UIs (when the requesting component isn't available, we use a default one temporarily).
- *Consistency*: Keeping the frontend state in sync with the backend can be difficult, especially in real-time applications or those that involve multiple users altering the same piece of data.
- *Caching*: Storing some remote state locally can improve performance but bring its own challenges, such as invalidation and staleness. In other words, if the remote data is altered by others, we need a mechanism to receive updates or perform a refetch to update our local state, which introduces a lot of complexity.
- *Updates and optimistic UI*: When a user makes a change, you can update the UI optimistically assuming the server call will succeed. But if it doesn't, you'll need a way to roll back those changes in your frontend state.

And those are only some of the challenges of remote states.

When the data is stored and accessible immediately in the frontend, you basically think in a linear way. This means you access and manipulate data in a straightforward sequence, one operation following

another, leading to a clear and direct flow of logic. This way of thinking aligns well with the synchronous nature of the code, making the development process intuitive and easier to follow.

Let's compare how much more code we'll need for rendering static data with remote data. Think about a famous quotes application that displays a list of quotes on the page.

To render the passed-in quotes list, you can map the data into JSX elements, like so:

```
function Quotes(quotes: string[]) {
  return (
    <ul>
      {quotes.map((quote, index) => <li key={index}>{quote}</li>)}
    </ul>
  );
}
```

Note

We're using `index` as the key here, which is fine for static quotes. However, it's generally best to avoid this practice. Using indices can lead to rendering issues in dynamic lists in real-world scenarios.

If the quotes are from a remote server, the code will turn into something like the following:

```
import React, { useState, useEffect } from 'react';

function Quotes() {
  const [quotes, setQuotes] = useState<string[]>([]);

  useEffect(() => {
    fetch('https://quote-service.com/quotes')
      .then(response => response.json())
      .then(data => setQuotes(data));
  }, []);

  return (
    <ul>
      {quotes.map((quote, index) => <li key={index}>{quote}</li>)}
    </ul>
  );
}

export default Quotes;
```

In this React component, we use `useState` to create a quotes state variable, initially set as an empty array. The `useEffect` Hook fetches quotes from a remote server when the component mounts. It then updates the quotes state with the fetched data. Finally, the component renders a list of quotes, iterating through the `quotes` array.

Don't worry, there's no need to sweat about the details for now; we'll delve into them in the next chapter on React essentials.

The previous code example shows the ideal scenario, but in reality, asynchronous calls come with their own challenges. We have to think about what to display while data is being fetched and how to handle various error scenarios, such as network issues or resource unavailability. These added complexities can make the code lengthier and more difficult to grasp.

For instance, while fetching data, we temporarily transition into a loading state, and should anything go awry, we shift to an error state:

```
function Quotes() {
  const [quotes, setQuotes] = useState<string[]>([]);
  const [isLoading, setIsLoading] = useState<boolean>(false);
  const [error, setError] = useState<Error | null>(null);

  useEffect(() => {
    setIsLoading(true);

    fetch('https://quote-service.com/quotes')
      .then(response => {
        if (!response.ok) {
          throw new Error('Failed to fetch quotes');
        }
        return response.json();
      })
      .then(data => {
        setQuotes(data);
      })
      .catch(err => {
        setError(err.message);
      })
      .finally(() => {
        setIsLoading(false);
      });
  }, []);

  return (
    <div>
      {isLoading && <p>Loading...</p>}
```

```
      {error && <p>Error: {error}</p>}
      <ul>
        {quotes.map((quote, index) => <li key={index}>{quote}</li>)}
      </ul>
    </div>
  );
}
```

The code uses `useState` to manage three pieces of state: `quotes` for storing the quotes, `isLoading` for tracking the loading status, and `error` for any fetch errors.

The `useEffect` Hook triggers the fetch operation. If the fetch is successful, the quotes are displayed and `isLoading` is set to `false`. If an error occurs, an error message is displayed and `isLoading` is again set to `false`.

As you can observe, the portion of the component dedicated to actual rendering is quite small (i.e., the JSX code inside `return`). In contrast, managing the state consumes nearly two-thirds of the function's body.

But that's just one aspect of the state management. There's also the matter of managing local state, which means the state only needs to be maintained inside a component. For example, as demonstrated in *Figure 1.3*, an accordion component needs to track whether it's expanded or collapsed – when you click the triangle on the header, it toggles the list panel:

Figure 1.3: An expandable section

Using a third-party state management library such as Redux or MobX can be beneficial when your application reaches a level of complexity that makes state tracking difficult. However, using a third-party state management library isn't without its caveats (learning curve, best practices in a particular library, migration efforts, etc.) and should be considered carefully. That's why many developers are leaning toward using React's built-in Context API for state management.

Another significant complexity in modern frontend applications that often goes unnoticed by many developers, yet is akin to an iceberg that warrants closer attention, is "unhappy paths." Let's look at these next.

Exploring "unhappy paths"

When it comes to UI development, our primary focus is often on the "happy path" – the optimal user journey where everything goes as planned. However, neglecting the "unhappy paths" can make your UI far more complicated than you might initially think. Here are some scenarios that could lead to unhappy paths and consequently complicate your UI development efforts.

Errors thrown from other components

Imagine that you're using a third-party component or even another team's component within your application. If that component throws an error, it could potentially break your UI or lead to unexpected behaviors that you have to account for. This can involve adding conditional logic or error boundaries to handle these errors gracefully, making your UI more complex than initially anticipated.

For example, in a `MenuItem` component that renders an item's data, let's see what happens when we try accessing something that doesn't exist in the passed-in prop `item` (in this case, we're looking for the aptly named `item.something.doesnt.exist`):

```
const MenuItem = ({
  item,
  onItemClick,
}: {
  item: MenuItemType;
  onItemClick: (item: MenuItemType) => void;
}) => {
  const information = item.something.doesnt.exist;

  return (
    <li key={item.name}>
      <h3>{item.name}</h3>
      <p>{item.description}</p>
      <button onClick={() => onItemClick(item)}>Add to Cart</button>
    </li>
  );
};
```

The `MenuItem` component receives an `item` object and an `onItemClick` function as props. It displays the item's name and description, as well as including an **Add to Cart** button. When the button is clicked, the `onItemClick` function is called with the item as an argument.

This code attempts to access a non-existing property, `item.something.doesnt.exist`, which will cause a runtime error. As demonstrated in *Figure 1.4*, the application stopped working after the backend service returned some unexpected data:

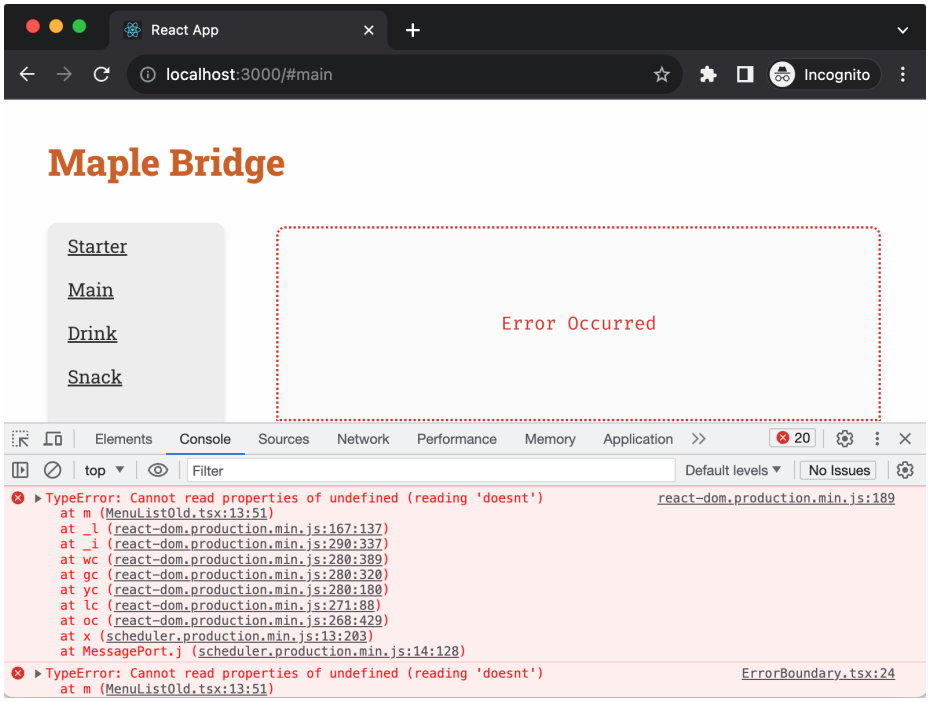

Figure 1.4: A component-thrown exception during render

This can cause the whole application to crash if we don't isolate the error into an **error boundary**, as we can see in *Figure 1.4* – the menus are not displayed, but the category and page titles remain functional; the area affected, which I've outlined with a red dotted line, is where the menus were supposed to appear. Error boundaries in React are a feature that allows you to catch JavaScript errors that occur in child components, log those errors, and display a fallback UI instead of letting the whole app crash. Error boundaries catch errors during rendering, in life cycle methods, and in constructors of the whole tree below them.

In real-world projects, your UI might depend on various microservices or APIs for fetching data. If any of these downstream systems are down, your UI has to account for it. You'll need to design fallbacks, loading indicators, or friendly error messages that guide the user on what to do next. Handling these scenarios effectively often involves both frontend and backend logic, thus adding another layer of complexity to your UI development tasks.

Learning the unexpected user behavior

No matter how perfectly you design your UI, users will always find ways to use your system in manners you didn't anticipate. Whether they input special characters in text fields, try to submit forms too quickly, or use browser extensions that interfere with your site, you have to design your UI to handle these edge cases. This means implementing additional validation, checks, and safeguards that can complicate your UI code base.

Let's examine a basic `Form` component to understand the considerations for user input. While this single-field form might require additional logic in the `handleChange` method, it's important to note that most forms typically consist of several fields (which means there will be more unexpected user behavior we need to consider):

```
import React, { ChangeEvent, useState } from "react";

const Form = () => {
  const [value, setValue] = useState<string>("");

  const handleChange = (event: ChangeEvent<HTMLInputElement>) => {
    const inputValue = event.target.value;
    const sanitizedValue = inputValue.replace(/[^\w\s]/gi, "");
    setValue(sanitizedValue);
  };

  return (
    <div>
      <form>
        <label>
          Input without special characters:
          <input type="text" value={value} onChange={handleChange} />
        </label>
      </form>
    </div>
  );
};

export default Form;
```

This `Form` component consists of a single text input field that restricts input to alphanumeric characters and spaces. It uses a `value` state variable to store the input field's value. The `handleChange` function, triggered on each input change, removes any non-alphanumeric characters from the user's input before updating the state with the sanitized value.

Understanding and effectively managing these unhappy paths are critical to creating a robust, resilient, and user-friendly interface. Not only do they make your application more reliable, but they also contribute to a more comprehensive and well-thought-out user experience.

I believe you should now have a clearer insight into the challenges of building modern frontend applications in React. Tackling these hurdles isn't straightforward, particularly since React doesn't offer a definitive guide on which approach to adopt, how to structure your code base, manage states, or ensure code readability (and by extension, ease of maintenance in the long run), or how established patterns can be of aid, among other concerns. This lack of guidance often leads developers to create solutions that might work in the short term but could be riddled with anti-patterns.

Exploring common anti-patterns in React

Within the realm of software development, we often encounter practices and approaches that, at first glance, appear to offer a beneficial solution to a particular problem. These practices, labeled as **anti-patterns**, may provide immediate relief or a seemingly quick fix, but they often hide underlying issues. Over time, reliance on these anti-patterns can lead to greater complexities, inefficiencies, or even the very issues they were thought to resolve.

Recognizing and understanding these anti-patterns is crucial for developers, as it enables them to anticipate potential pitfalls and steer clear of solutions that may be counterproductive in the long run. In the upcoming sections, we'll highlight common anti-patterns accompanied by code examples. We'll address each anti-pattern and outline potential solutions. However, we won't delve deeply here since entire chapters are dedicated to discussing these topics in detail.

Props drilling

In complex React applications, managing state and ensuring that every component has access to the data it needs can become challenging. This is often observed in the form of **props drilling**, where props are passed from a parent component through multiple intermediary components before they reach the child component that actually needs them.

For instance, consider a `SearchableList`, `List`, and a `ListItem` hierarchy – a `SearchableList` component contains a `List` component, and `List` contains multiple instances of `ListItem`:

```
function SearchableList({ items, onItemClick }) {
  return (
    <div className="searchable-list">
      {/* Potentially some search functionality here */}
      <List items={items} onItemClick={onItemClick} />
    </div>
  );
}

function List({ items, onItemClick }) {
  return (
    <ul className="list">
      {items.map(item => (
        <ListItem key={item.id} data={item} onItemClick={onItemClick}
         />
      ))}
    </ul>
  );
}

function ListItem({ data, onItemClick }) {
```

```
  return (
    <li className="list-item" onClick={() => onItemClick(data.id)}>
      {data.name}
    </li>
  );
}
```

In this setup, the `onItemClick` prop is drilled from `SearchableList` through `List` and finally to `ListItem`. Though the `List` component doesn't use this prop, it has to pass it down to `ListItem`.

This approach can lead to increased complexity and reduced maintainability. When multiple props are passed down through various components, understanding the flow of data and debugging can become difficult.

A potential solution to avoid props drilling in React is by leveraging the Context API. It provides a way to share values (data and functions) between components without having to explicitly pass props through every level of the component tree.

In-component data transformation

The component-centric approach in React is all about breaking up tasks and concerns into manageable chunks, enhancing maintainability. One recurrent misstep, however, is when developers introduce complex data transformation logic directly within components.

It's common, especially when dealing with external APIs or backends, to receive data in a shape or format that isn't ideal for the frontend. Instead of adjusting this data at a higher level, or in a utility function, the transformation is defined inside the component.

Consider the following scenario:

```
function UserProfile({ userId }) {
  const [user, setUser] = useState(null);

  useEffect(() => {
    fetch(`/api/users/${userId}`)
      .then(response => response.json())
      .then(data => {
        // Transforming data right inside the component
        const transformedUser = {
          name: `${data.firstName} ${data.lastName}`,
          age: data.age,
          address: `${data.addressLine1}, ${data.city}, ${data.
           country}`
        };
        setUser(transformedUser);
```

```
      });
  }, [userId]);

  return (
    <div>
      {user && (
        <>
          <p>Name: {user.name}</p>
          <p>Age: {user.age}</p>
          <p>Address: {user.address}</p>
        </>
      )}
    </div>
  );
}
```

The `UserProfile` function component retrieves and displays a user's profile based on the provided prop `userId`. Once the remote `data` is fetched, it's transformed within the component itself to create a structured user profile. This transformed data consists of the user's full name (a combination of first and last name), age, and a formatted address.

By directly embedding the transformation, we encounter a few issues:

- *Lack of clarity*: Combining data fetching, transformation, and rendering tasks within a single component makes it harder to pinpoint the component's exact purpose
- *Reduced reusability*: Should another component require the same or a similar transformation, we'd be duplicating logic
- *Testing challenges*: Testing this component now requires considering the transformation logic, making tests more convoluted

To combat this anti-pattern, it's advised to separate data transformation from the component. This can be achieved using utility functions or custom Hooks, thus ensuring a cleaner and more modular design. By externalizing these transformations, components remain focused on rendering and business logic stays centralized, making for a far more maintainable code base.

Complicated logic in views

The beauty of modern frontend frameworks, including React, is the distinct separation of concerns. By design, components should be oblivious to the intricacies of business logic, focusing instead on presentation. However, a recurrent pitfall that developers encounter is the infusion of business logic within view components. This not only disrupts the clean separation but also bloats components and makes them harder to test and reuse.

Consider a simple example. Imagine a component that is meant to display a list of items fetched from an API. Each item has a price, but we want to display items above a certain threshold price:

```
function PriceListView({ items }) {
  // Business logic within the view
  const filterExpensiveItems = (items) => {
    return items.filter(item => item.price > 100);
  }

  const expensiveItems = filterExpensiveItems(items);

  return (
    <div>
      {expensiveItems.map(item => (
        <div key={item.id}>
          {item.name}: ${item.price}
        </div>
      ))}
    </div>
  );
}
```

Here, the `filterExpensiveItems` function, a piece of business logic, resides directly within the view component. The component is now tasked with not just presenting data but also processing it.

This approach can become problematic:

- *Reusability*: If another component requires a similar filter, the logic would need to be duplicated
- *Testing*: Unit testing becomes more complex as you're not just testing the rendering, but also the business logic
- *Maintenance*: As the application grows and more logic is added, this component can become unwieldy and harder to maintain

To ensure our components remain reusable and easy to maintain, it's wise to embrace the **separation of concerns** principle. This principle states that each module or function in software should have responsibility over a single part of the application's functionality. By separating the business logic from the presentation layer and adopting a **layered architecture**, we can ensure each part of our code handles its own specific responsibility, leading to a more modular and maintainable code base.

Lack of tests

Imagine building a shopping cart component for an online store. The cart is crucial as it handles item additions, removals, and total price calculations. As straightforward as it may seem, it embodies various

moving parts and logic interconnections. Without tests, you leave the door open for future problems, such as incorrect pricing, items not being added or removed correctly, or even security vulnerabilities.

Consider this simplistic version of a shopping cart:

```
function ShoppingCart() {
  const [items, setItems] = useState([]);

  const addItem = (item) => {
    setItems([...items, item]);
  };

  const removeItem = (itemId) => {
    setItems(items.filter(item => item.id !== itemId));
  };

  const calculateTotal = () => {
    return items.reduce((total, item) => total + item.price, 0);
  };

  return (
    <div>
      {/* Render items and controls for adding/removing */}
      <p>Total: ${calculateTotal()}</p>
    </div>
  );
}
```

While this shopping cart's logic appears straightforward, potential pitfalls are lurking. What if an item gets added multiple times erroneously, or prices change dynamically, or discounts are applied? Without tests, these scenarios might not be evident until a user encounters them, which could be detrimental to the business.

Enter **test-driven development** (**TDD**). TDD emphasizes writing tests before the actual component or logic. For our `ShoppingCart` component, it means having tests verifying that items are correctly added or removed, total calculations are adjusted appropriately, and edge cases, such as handling discounts, are managed. Only after these tests are in place should the actual component logic be implemented. TDD is more than just catching errors early; it champions well-structured, maintainable code.

For the `ShoppingCart` component, adopting TDD would necessitate tests ensuring items get added or removed as expected, totals are correctly computed, and edge cases are tackled seamlessly. This way, as the application grows, the foundational TDD tests ensure each modification or addition maintains the application's integrity and correctness.

Duplicated code

It's a familiar sight in many code bases – chunks of identical or very similar code scattered across different parts of the application. Duplicated code not only bloats the code base but also introduces potential points of failure. When a bug is detected or an enhancement is needed, every instance of the duplicated code may need to be altered, leading to an increased likelihood of introducing errors.

Let's consider two components in which the same filtering logic is repeated:

```
function AdminList(props) {
  const filteredUsers = props.users.filter(user => user.isAdmin);
  return <List items={filteredUsers} />;
}

function ActiveList(props) {
  const filteredUsers = props.users.filter(user => user.isActive);
  return <List items={filteredUsers} />;
}
```

The **DRY** (**don't repeat yourself**) **principle** comes to the rescue here. By centralizing common logic into utility functions or **higher-order components** (**HOCs**), the code becomes more maintainable and readable, and less prone to errors. For this example, we could abstract the filtering logic and reuse it, ensuring a singular source of truth and easier updates.

Long component with too much responsibility

React encourages the creation of modular, reusable components. However, as features get added, a component can quickly grow in size and responsibility, turning into an unwieldy behemoth. A long component that manages various tasks becomes difficult to maintain, understand, and test.

Imagine an `OrderContainer` component that has a huge prop list that includes a lot of different aspects of the responsibilities:

```
const OrderContainer = ({
  testID,
  orderData,
  basketError,
  addCoupon,
  voucherSelected,
  validationErrors,
  clearErrors,
  removeLine,
  editLine,
  hideOrderButton,
  hideEditButton,
```

```
  loading,
}: OrderContainerProps) => {
  //..
}
```

Such a component violates the **single-responsibility principle** (**SRP**), which advocates that a component should fulfill only one function. By taking on multiple roles, it becomes more complex and less maintainable. We need to analyze the core responsibility of the `OrderContainer` component and separate the supporting logic into other smaller, focused components or utilize Hooks for logic separation.

> **Note**
>
> These listed anti-patterns have different variations, and we'll discuss the solutions correspondingly in the following chapters. Apart from that, there are also some more generic design principles and design patterns we'll discuss in the book, as well as some proven engineering practices, such as refactoring and TDD.

Unveiling our approach to demolishing anti-patterns

When it comes to addressing prevalent anti-patterns, an arsenal of design patterns comes to the fore. Techniques such as **render props**, HOCs, and **Hooks** are instrumental in augmenting component capabilities without deviating from their primary roles, while leveraging foundational patterns such as layered architecture and separation of concerns ensures a streamlined code base, demarcating logic, data, and presentation in a coherent manner. Such practices don't just elevate the sustainability of React apps but also lay the groundwork for effective teamwork among developers.

Meanwhile, **interface-oriented programming**, at its core, zeroes in on tailoring software centered around the interactions occurring between software modules, predominantly via interfaces. Such a modus operandi fosters agility, rendering software modules not only more coherent but also amenable to alterations. The **headless components** paradigm, on the other hand, embodies components that, while devoid of direct rendering duties, are entrusted with the management of state or logic. These components pass the baton to their consuming counterparts for UI rendering, thus championing adaptability and reusability.

By gaining a firm grasp on these design patterns and deploying them judiciously, we're positioned to circumvent prevalent missteps, thereby uplifting the stature of our React applications.

Plus, within the coding ecosystem, the twin pillars of TDD and consistent refactoring emerge as formidable tools to accentuate code quality. TDD, with its clarion call of test-before-code, furnishes an immediate feedback loop for potential discrepancies. Hand-in-hand with TDD, the ethos of persistent refactoring ensures that code is perpetually optimized and honed. Such methodologies not only set the benchmark for code excellence but also instill adaptability to forthcoming changes.

As we navigate the realm of refactoring, it's pivotal to delve into the essence of these techniques, discerning their intricacies and optimal application points. Harnessing these refactoring avenues promises to bolster your code's clarity, sustainability, and overarching efficiency. This is something that we'll be doing throughout the book!

Summary

In this chapter, we explored the challenges of UI development from its complexities to state management issues. We also discussed the common anti-patterns due to the nature of its complexity, and briefly introduced our approach that combines best practices and effective testing strategies. This sets the foundation for more efficient and robust frontend development.

In the upcoming chapter, we'll dive deep into React essentials, giving you the tools and knowledge you need to master this powerful library. Stay tuned!

2

Understanding React Essentials

Welcome to this fundamental chapter of our React essentials guide! This chapter serves as a solid foundation for your journey into the exciting world of React development. We will delve into the fundamental concepts of React and provide you with the essential knowledge needed to kickstart your React projects with confidence.

In this chapter, we will explore how to think in components, a crucial mindset for building reusable and modular **user interfaces** (**UIs**). You will learn the art of breaking your application down into smaller, self-contained components, enabling you to create maintainable and scalable code bases. By understanding this fundamental concept, you will be equipped with the skills to architect robust and flexible React applications.

Additionally, we will introduce you to the most commonly used Hooks in React, such as `useState`, `useEffect`, and more. These Hooks are powerful tools that allow you to manage state, handle side effects, and tap into React's lifecycle methods within functional components. By mastering these Hooks, you will have the ability to create dynamic and interactive UIs effortlessly.

By the end of this chapter, you will be well prepared to explore more advanced topics and tackle real-world React challenges in the subsequent chapters of our guide. So, buckle up and get ready to embark on an exciting journey into the world of React.

So, in this chapter, we will cover the following topics:

- Understanding static components in React
- Creating components with props
- Breaking down UIs into components
- Managing internal state in React
- Understanding the rendering process
- Exploring common React Hooks

Technical requirements

A GitHub repository has been created to host all the code we discuss in the book. For this chapter, you can find it under `https://github.com/PacktPublishing/React-Anti-Patterns/tree/main/code/src/ch2`.

Understanding static components in React

React applications are built on components. A **component** can range from a simple function returning an HTML snippet to a more complex one that interacts with network requests, dynamically generates HTML tags, and even auto-refreshes based on backend service changes.

Let's start with a basic scenario and define a **static component**. In React, a static component (also known as presentational components or dumb components) refers to a component that doesn't have any state and doesn't interact with the data or handle any events. It is a component that only renders the UI based on the props it receives. Here's an example:

```
const StaticArticle = () => {
  return (
    <article>
      <h3>Think in components</h3>
      <p>It's important to change your mindset when coding with
       React.</p>
    </article>
  );
};
```

This static component closely resembles the corresponding HTML snippet, which uses the `<article>` tag to structure content with a title and a paragraph:

```
<article>
  <h3>Think in components</h3>
  <p>It's important to change your mindset when coding with React.</p>
</article>
```

While encapsulating static HTML in a component is useful, there may be a need for the component to represent different articles, not just a specific one. Just as we pass parameters to a function to make it more versatile, we can pass parameters into a component to make it useful in different contexts. That can be done with props, which we will discuss in the next section.

Creating components with props

In React, components can receive input in the form of properties, which are commonly referred to as props. **Props** allow us to pass data from a parent component to its child components. This mechanism enables components to be reusable and adaptable, as they can receive different sets of props to customize their behavior and appearance.

Props are essentially JavaScript objects containing key-value pairs, where the keys represent the prop names and the values contain the corresponding data. These props can include various types of data, such as strings, numbers, Booleans, or even functions.

By passing props to a component, we can control its rendering and behavior dynamically. This allows us to create flexible and composable components that can be easily composed together to build complex UIs.

Now, let's move beyond a static component and see how we can make it more generic by using props. Suppose we want to display a list of blog posts, each with a heading and a summary. In HTML, we would manually write the HTML fragments. However, with React components, we can dynamically generate these HTML fragments using JavaScript.

First, let's start with the basic structure:

```
type ArticleType = {
  heading: string;
  summary: string;
};

const Article = ({ heading, summary }: ArticleType) => {
  return (
    <article>
      <h3>{heading}</h3>
      <p>{summary}</p>
    </article>
  );
};
```

We can then pass the desired `heading` and `summary` values to the `Article` component:

```
<Article
  heading="Think in components"
  summary="It's important to change your mindset when coding with
React."
/>
```

Or, we could define another `Article` component:

```
<Article
  heading="Define custom hooks"
  summary="Hooks are a great way to share state logic."
/>
```

By using props, we can pass different values to the `heading` and `summary` props when we use the `Article` component. This makes the component versatile and reusable, as it can be used to display various articles with different titles and summaries based on the provided props.

Props are a fundamental concept in React that allows us to customize and configure components, making them dynamic and adaptable to different data or requirements.

A component can have any number of props, although it's recommended to keep them to a manageable amount, preferably no more than five or six. This helps maintain clarity and understandability, as having too many props can make the component harder to comprehend and extend.

Breaking down UIs into components

Let's examine a more complex UI and explore how to break it down into components and implement them separately. In this example, we will use a weather application:

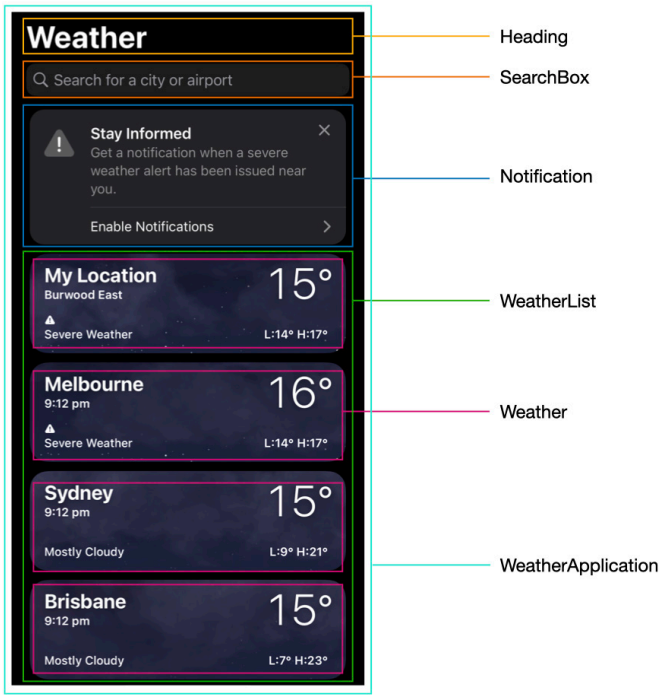

Figure 2.1: A weather application

The entire application can be defined as a `WeatherApplication` component, which includes several subcomponents:

```
const WeatherApplication = () => {
  return (
    <>
      <Heading title="Weather" />
      <SearchBox />
```

```
      <Notification />
      <WeatherList />
    </>
  );
};
```

Each subcomponent can perform various tasks, such as fetching data from a remote server, conditionally rendering a drop-down list, or auto-refreshing periodically.

For example, a `SearchBox` component might have the following structure:

```
const SearchBox = () => {
  return (
    <div className="search-box">
      <input type="text" />
      <button>Search</button>
      <div className="search-results" />
    </div>
  );
};
```

The `search-results` section will only appear when data is fetched from the search query.

On the other hand, a `Weather` component could be more straightforward, rendering whatever is passed to it:

```
type WeatherType = {
  cityName: string;
  temperature: number;
  weather: string;
};

const Weather = ({ cityName, temperature, weather }: WeatherType) => {
  return (
    <div>
      <span>{cityName}</span>
      <span>{temperature}</span>
      <span>{weather}</span>
    </div>
  );
};
```

When implementing components in real-world scenarios, it is crucial to pay attention to styling and refine the HTML structure with meticulous detail. Additionally, components should effectively manage their own state, ensuring consistency and responsiveness across renders.

By grasping the concept of components and their proper structuring in React, you gain the ability to construct dynamic and reusable UI elements that contribute to the overall functionality and organization of your application.

As you advance in your React development journey, don't forget to polish the visual presentation through styling and consider efficient state management techniques to enhance the performance and interactivity of your components.

A complete `Weather` component could be something like this:

```
type Weather = {
  main: string;
  temperature: number;
}

type WeatherType = {
  name: string;
  weather: Weather;
}

export function WeatherCard({ name, weather }: WeatherType) {
  return (
    <div className={`weather-container ${weather.main}`}>
      <h3>{name}</h3>
      <div className="details">
        <p className="temperature">{weather.temperature}</p>
        <div className="weather">
          <span className="weather-category">{weather.main}</span>
        </div>
      </div>
    </div>
  );
}
```

The code snippet defines two types: `Weather` and `WeatherType`. The `Weather` type represents the weather data with two properties: `main` (string) and `temperature` (number). The `WeatherType` type represents the structure of the weather data for a specific location, with a `name` property (string) for the location name and a `weather` property of the `Weather` type.

The `WeatherCard` component receives the name and weather props of the `WeatherType` type. Inside the component, it renders a `div` container with a dynamic class based on the `weather.main` value.

When working on complex UIs, it is crucial to break them down into smaller, manageable components. Each component should represent a separate concept, and they can be combined together using **JSX**

(**JavaScript Extension**), similar to writing HTML code. While props are useful for passing data to components, there are situations where we need to maintain data within a component itself. This is where states come into play, allowing us to manage and update data internally.

Managing internal state in React

State in React refers to the internal data that components can hold and manage. It allows components to store and update information, enabling dynamic UI updates, interactivity, and data persistence. State is a fundamental concept in React that helps build responsive and interactive applications.

Applications feature various state types, such as a Boolean for toggle status, a loading state for network requests, or a user-input string for queries. We'll explore the `useState` Hook, ideal for maintaining local state within a component across re-renders. React **Hooks** are a feature introduced in React 16.8 that enables functional components to have state and lifecycle features (we'll discuss Hooks in more detail in the *Exploring common React Hooks* section later).

Let's begin by using a simple Hook for internal state management to understand how it maintains data within a component. For example, the following `SearchBox` component can be implemented with something like this:

```
const SearchBox = () => {
  const [query, setQuery] = useState<string>();
  const handleChange = (e: ChangeEvent<HTMLInputElement>) => {
    const value = e.target.value;
    setQuery(value);
  };

  return (
    <div className="search-box">
      <input type="text" value={query} onChange={handleChange} />
      <button>Search</button>
      <div className="search-results">{query}</div>
    </div>
  );
};
```

The code snippet showcases that `SearchBox` includes an input field, a search button, and a display area for search results. The `useState` Hook is used to create a state variable called `query`, initialized as an empty string. The `handleChange` function captures the user's input and updates the query state accordingly. Then, `component` renders the input field with the current value of `query`, a search button, and a `div` tag as a container displaying the search results.

Note here the `useState` Hook is used to manage state within the `SearchBox` component:

```
const [query, setQuery] = useState<string>("");
```

Let's explain this code a little more specifically:

- `useState<string>("")`: This line declares a state variable called query and initializes it with an empty string (`""`)
- `const [query, setQuery]`: This syntax uses array destructuring to assign the state variable (`query`) and its corresponding update function (`setQuery`) to variables with the same names

Now, we have the state and the setter function bound to the input box. As we enter a city name in the input box, the `search-results` area is automatically updated, along with the input value. Although there are multiple re-renders occurring as we type, the state value persists throughout the process:

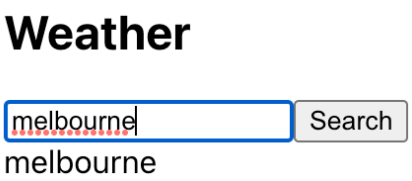

Figure 2.2: Managing state with useState

The `useState` Hook is excellent for managing internal state, yet real-world projects often require handling various other state types. As applications expand, managing global-level data shared across multiple components, such as from parent to children, becomes necessary. We'll discuss different mechanisms for this kind of state management in later sections.

Now that we have gained an understanding of how props and state enable us to create dynamic components, it is important to explore how changes in data impact the rendering process in React. By understanding this process, we can take steps to optimize our code for improved efficiency and performance.

Understanding the rendering process

When the data that a React component relies on changes, whether it is through updated props or a modified state, React needs to update the UI to reflect those changes. The process is called **rendering**, and is composed of the following steps:

- **Initial render**: When a functional component is first rendered, it generates a virtual representation of the component's UI. This virtual representation describes the structure and content of the UI elements.
- **State and props changes**: When there are changes in the component's state or props, React re-evaluates the component's function body. It performs a diffing algorithm to compare the previous and new function bodies, identifying the differences between them.

> **Note**
>
> In React, a **diffing** algorithm is an internal mechanism that compares previous and new virtual **Document Object Model** (**DOM**) representations of a component and determines the minimal set of changes needed to update the actual DOM.

- **Reconciliation**: React determines which parts of the UI need to be updated based on the differences identified during the diffing process. It updates only those specific parts of the UI, keeping the rest unchanged.
- **Re-rendering**: React re-renders the component by updating the virtual representation of the UI. It generates a new virtual DOM based on the updated function body, replacing the previous virtual DOM.
- **DOM update**: Finally, React efficiently updates the real DOM to reflect the changes in the virtual DOM. It applies the necessary DOM manipulations, such as adding, removing, or updating elements, to make the UI reflect the updated state and props.

This process ensures that the UI remains in sync with the component's state and props, enabling a reactive and dynamic UI. React's efficient rendering approach minimizes unnecessary DOM manipulations and provides a performant rendering experience in functional components.

In this book, we will explore situations where writing high-performance code is crucial, ensuring that components only re-render when necessary while preserving unchanged parts. Achieving this requires utilizing Hooks effectively and employing various techniques to optimize rendering.

In an application, data management is essential, but we also encounter side effects such as network requests, DOM events, and the need for data sharing among components. To tackle these challenges, React provides a range of commonly used Hooks that serve as powerful tools for building applications. Let's explore these Hooks and see how they can greatly assist us in our development process.

Exploring common React Hooks

We briefly talked about Hooks in the *Managing internal state in React* section. In addition, Hooks allow for code reuse, improved readability, and easier testing by separating concerns and making component logic more modular.

Let's discuss a few of the most common Hooks in this section. Please be aware that in this chapter, we are focusing on the most used Hooks. As we progress through the book, we will delve into several more advanced applications of these Hooks. Regarding the last Hook, `useContext`, we will initially explore its basic usage at the end of this section to provide an introductory understanding. In later chapters, we'll employ `useContext` in more complex scenarios, allowing for a deeper and more practical grasp of its functionality.

useState

We have already seen the basic usage of `useState` Hook previously in this chapter. You can define as many states as you like inside a component, and it's quite common to do so in real-world projects. For example, a login form might include a username, a password, and a **Remember Me** flag. All these states need to be remembered before the user clicks the submit (**Login**) button:

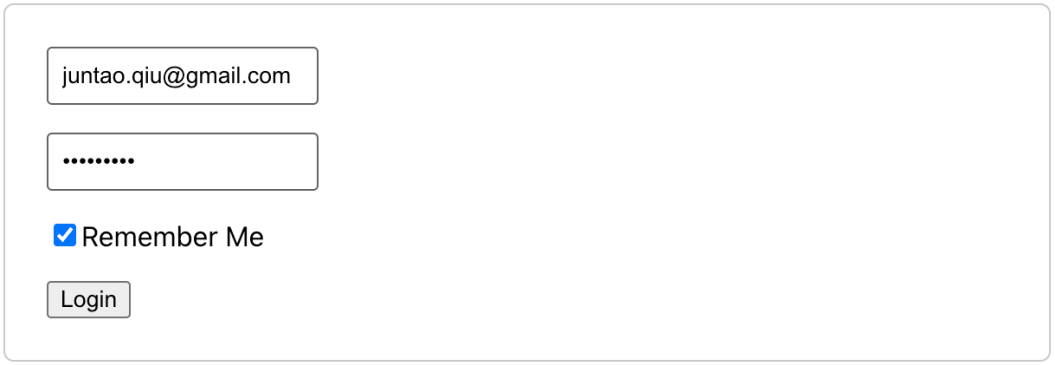

Figure 2.3: Login form

Based on the UI, we'll need three different states for the username, password, and a Boolean flag for **Remember Me**, like so:

```
const Login = () => {
  const [username, setUsername] = useState<string>("");
  const [password, setPassword] = useState<string>("");
  const [rememberMe, setRememberMe] = useState<boolean>(false);

  return (
    <div className="login-form">
      <div className="field">
        <input
          type="text"
          value={username}
          onChange={(event) => setUsername(event.target.value)}
          placeholder="Username"
        />
      </div>

      <div className="field">
        <input
          type="password"
          value={password}
```

```
            onChange={(event) => setPassword(event.target.value)}
            placeholder="Password"
          />
        </div>

        <div className="field">
          <label>
            <input
              type="checkbox"
              checked={rememberMe}
              onChange={(event) => setRememberMe(event.target.checked)}
            />
            Remember Me
          </label>
        </div>

        <div className="field">
          <button>Login</button>
        </div>
      </div>
    );
};
```

In the code snippet, we have to manage three different states, so it uses the `useState` Hook to manage state for the `username`, `password`, and `rememberMe` fields. The component renders input fields for username and password, a checkbox for **Remember Me**, and a **Login** button. User input updates the corresponding state variables, enabling the capture of form data.

useEffect

In React, a side effect refers to any code that is not directly related to rendering a component but has an impact outside the component's scope. Side effects often involve interacting with external resources, such as making API calls, modifying the underlying DOM (not using the normal React virtual DOM), subscribing to event listeners, or managing timers.

React provides a built-in Hook called `useEffect` to handle side effects within functional components. The `useEffect` Hook allows you to perform side effects after rendering or when specific dependencies change.

By using the `useEffect` Hook, you can ensure that side effects are executed at the appropriate times during the component's lifecycle. This helps maintain the consistency and integrity of the application while separating side effects from the core rendering logic.

Let's take a look at a typical use case of `useEffect`. We created an `Article` component in the *Creating components with props* section; now, let's make a list of articles. Normally, an article list could return from some API calls; in JSON format, for example. We can use the `useEffect` Hook to send the request and set state once the response is returned from an API call, like so:

```
const ArticleList = () => {
  const [articles, setArticles] = useState<ArticleType[]>([]);

  useEffect(() => {
    const fetchArticles = async () => {
      fetch("/api/articles")
        .then((res) => res.json())
        .then((data) => setArticles(data));
    };

    fetchArticles();
  }, []);

  return (
    <div>
      {articles.map((article) => (
        <Article heading={article.heading} summary={article.summary}
         />
      ))}
    </div>
  );
};
```

The code snippet demonstrates the usage of the `useEffect` Hook in a React functional component. Let's break it down:

- `useEffect(() => { ... }, []);`: This line declares the `useEffect` Hook and provides two arguments. The first argument is a callback function that contains the side effect code to be executed. The second argument is an array of dependencies that determines when the side effect should be triggered. An empty array, `[]`, indicates that the side effect should only run once during the initial render.
- `const fetchArticles = async () => { ... }`: This line declares an asynchronous function called `fetchArticles`. Inside this function, an API call is made to fetch data from the `/api/articles` endpoint.
- `fetch("/api/articles")...`: This line uses the `fetch` function to make a `GET` request to the specified API endpoint. The response is then processed using promises (`then`) to extract the JSON data.

- `setArticles(data)`: This line updates the component's state variable articles with the retrieved data using the `setArticles` function. This will trigger a re-render of the component with the updated data.
- `fetchArticles()`: This line invokes the `fetchArticles` function, triggering the API call and updating the article's state.

Once we have these articles, we can then use the `array.map` collection API to generate a list of articles.

It's worth mentioning that when strict mode is on, in development, React runs setup and cleanup one extra time before the actual setup. In practice, you can wrap the whole application inside the `StrictMode` built-in component from React, and your components will re-render an extra time to find bugs caused by impure rendering along with other checks.

Also, note that the second parameter of `useEffect` is critical. We used an empty array previously as we don't want the effect to trigger each time, but there are cases we would like to perform the effect whenever one of the dependencies changes.

For example, let's say we have an `ArticleDetail` component, and whenever the `id` prop of the article changes, we need to re-fetch the data and re-render:

```
const ArticleDetail = ({ id }: { id: string }) => {
  const [article, setArticle] = useState<ArticleType>();

  useEffect(() => {
    const fetchArticleDetail = async (id: string) => {
      fetch(`/api/articles/${id}`)
        .then((res) => res.json())
        .then((data) => setArticle(data));
    };

    fetchArticleDetail(id);
  }, [id]);

  return (
    <div>
      {article && (
        <Article heading={article.heading} summary={article.summary}
         />
      )}
    </div>
  );
};
```

Inside the `useEffect` Hook, the `fetchArticleDetail` function is defined to handle the API call. It fetches the article details based on the provided `id` prop, converts the response to JSON, and updates the article state using `setArticle`.

The effect is triggered when the `id` prop changes. Upon successful retrieval of the article data, the `Article` component is rendered with the `heading` and `summary` properties from the article state.

An essential feature of the `useEffect` Hook for handling side effects is the cleanup mechanism. When using `useEffect`, it's recommended to return a cleanup function that React will call upon the component's unmounting. For instance, if you set up a timer within `useEffect`, you should provide a function that clears this timer as the return value. This ensures proper resource management and prevents potential memory leaks in your application.

For example, let's say we have a component that needs to execute an effect 1 second after the initial rendering, as seen here:

```
const Timer = () => {
  useEffect(() => {
    const timerId = setTimeout(() => {
      console.log("time is up")
    }, 1000);

    return () => {
      clearTimeout(timerId)
    }
  }, [])

  return <div>Hello timer</div>
}
```

So, within the `Timer` component, the `useEffect` Hook is used to handle a side effect. As the component mounts, a `setTimeout` function is set up to log the message `time is up` after a delay of `1000` milliseconds. The `useEffect` Hook then returns a cleanup function to prevent memory leaks. This cleanup function uses `clearTimeout` to clear the timer identified by `timerId` when the component unmounts.

For the `ArticleDetail` example, the complete version with the cleanup function would be something like this:

```
useEffect(() => {
  const controller = new AbortController();
  const signal = controller.signal;

  const fetchArticleDetail = async (id: string) => {
    fetch(`/api/articles/${id}`, { signal })
```

```
        .then((res) => res.json())
        .then((data) => setArticle(data));
    };

    fetchArticleDetail(id);

    return () => {
      controller.abort();
    };
  }, [id]);
```

In this code snippet, an `AbortController` component is used within a `useEffect` Hook to manage the lifecycle of a network request. When the component mounts, the `useEffect` Hook triggers, creating a new instance of `AbortController` and extracting its signal. This signal is passed to the `fetch` function, linking the request to the controller.

If the component unmounts before the request completes, the cleanup function is called, using the `abort` method of the controller to cancel the ongoing fetch request. This prevents potential issues such as updating the state of an unmounted component, ensuring better performance and avoiding memory leaks.

Let's now turn our attention to another crucial Hook that enhances performance by preventing unnecessary function creation during re-renders.

useCallback

The `useCallback` Hook in React is used to memoize and optimize the creation of callback functions. It is particularly useful when passing callbacks to child components or when using callbacks as dependencies in other Hooks. You can see it in action here:

```
const memoizedCallback = useCallback(callback, dependencies);
```

The `useCallback` Hook takes two arguments:

- `callback`: This is the function that you want to memoize. It can be an inline function or a function reference.
- `dependencies`: This is an array of dependencies that the memoized callback depends on. If any of the dependencies change, the callback will be recreated.

Let's explore a practical example. We require an editor component to modify the summary of an article. Whenever a user types a character, the summary needs to be updated, triggering a re-render. However, this re-rendering can result in the creation of a new function each time, which can impact

performance. To mitigate this, we can utilize the `useCallback` Hook to optimize the rendering process and avoid unnecessary function recreations:

```
const ArticleEditor = ({ id }: { id: string }) => {
  const submitChange = useCallback(
    async (summary: string) => {
      try {
        await fetch(`/api/articles/${id}`, {
          method: "POST",
          body: JSON.stringify({ id, summary }),
          headers: {
            "Content-Type": "application/json",
          },
        });
      } catch (error) {
        // handling errors
      }
    },
    [id]
  );

  return (
    <div>
      <ArticleForm onSubmit={submitChange} />
    </div>
  );
};
```

In the `ArticleEditor` component, `useCallback` is used to memoize the `submitChange` function, which asynchronously makes a `POST` request to update an article, using the `fetch` API. This optimization with `useCallback` ensures that `submitChange` is only recreated when the `id` prop changes, enhancing performance by reducing unnecessary recalculations.

The component then renders `ArticleForm`, passing `submitChange` as a prop for handling form submissions:

```
const ArticleForm = ({ onSubmit }: { onSubmit: (summary: string) =>
void }) => {
  const [summary, setSummary] = useState<string>("");

  const handleSubmit = (e: FormEvent) => {
    e.preventDefault();
    onSubmit(summary);
  };

  const handleSummaryChange = useCallback(
```

```
    (event: ChangeEvent<HTMLTextAreaElement>) => {
      setSummary(event.target.value);
    },
    []
  );

  return (
    <form onSubmit={handleSubmit}>
      <h2>Edit Article</h2>
      <textarea value={summary} onChange={handleSummaryChange} />
      <button type="submit">Save</button>
    </form>
  );
};
```

`ArticleForm` uses the `useState` Hooks to track the summary state. When the form is submitted, `handleSubmit` prevents the default form action and calls `onSubmit` with the current summary. The `handleSummaryChange` function, optimized with `useCallback`, updates the summary state based on the `textarea` input. This use of `useCallback` ensures the function doesn't get recreated unnecessarily on each render, improving performance.

The React Context API

The **React Context API** is a feature that allows you to pass data directly through the component tree, without having to pass props down manually at every level. This comes in handy when your application has global data that many components share, or when you have to pass data through components that don't necessarily need the data but have to pass it down to their children.

For example, imagine that we are creating an application that includes a dark or light theme, depending on the current time (for instance, if it's daytime, we use light mode). We would need to set the theme at the root level.

So, first, we define a type `ThemeContextType` and create a context instance `ThemeContext` of the type:

```
import React from "react";

export type ThemeContextType = {
  theme: "light" | "dark";
};

export const ThemeContext = React.createContext<ThemeContextType |
undefined>(
  undefined
);
```

Then, we create a `ThemeProvider` component, which will use a React state to manage the current theme:

```
import React, { useState } from "react";
import { ThemeContext, ThemeContextType } from "./ThemeContext";

export const ThemeProvider = ({ children }) => {
  const [theme, setTheme] = useState<"light" | "dark">("light");

  const value: ThemeContextType = { theme };

  return (
    <ThemeContext.Provider value={value}>{children}</ThemeContext.
Provider>
  );
};
```

Finally, we can use the `ThemeProvider` component in our application:

```
import React from "react";
import { ThemeProvider } from "./ThemeProvider";
import App from "./App";

const Root = () => {
  return (
    <ThemeProvider>
      <App />
    </ThemeProvider>
  );
};

export default Root;
```

In any component in our application, we can now access the current theme:

```
import React, { useContext } from "react";
import { ThemeContext } from "./ThemeContext";

const ThemedComponent = () => {
  const context = useContext(ThemeContext);

  const { theme } = context;

  return <div className={theme}>Current Theme: {theme}</div>;
```

```
};

export default ThemedComponent;
```

In this setup, `ThemeContext` provides the current theme to any components in the tree that are interested in it. The theme is stored in a state variable in the `ThemeProvider` component, which is the root component of the app.

The provided code may not offer much utility since the theme cannot be modified. However, by utilizing the Context API, you can define a modifier that allows children nodes to alter the status. This mechanism proves highly beneficial for data sharing, making it a valuable tool. Let's modify the context interface a little bit:

```
type Theme = {
  theme: "light" | "dark";
  toggleTheme: () => void;
};

const ThemeContext = React.createContext<Theme>({
  theme: "light",
  toggleTheme: () => {},
});
```

We added a `toggleTheme` function in the context so that the component can modify the `theme` value when needed.

To implement the provider, we can utilize the `useState` Hook to define an internal state. By exposing the setter function, the children components can utilize it to update the value of the theme:

```
const ThemeProvider = ({ children }: { children: ReactNode }) => {
  // default theme is light
  const [theme, setTheme] = useState<"light" | "dark">("light");

  const toggleTheme = useCallback(() => {
    setTheme((prevTheme) => (prevTheme === "light" ? "dark" :
"light"));
  }, []);

  return (
    <ThemeContext.Provider value={{ theme, toggleTheme }}>
      {children}
    </ThemeContext.Provider>
  );
};
```

And then, in the calling site, it's straightforward to use the `useContext` Hook to access values in the context:

```
const Article = ({ heading, summary }: ArticleType) => {
  const { theme, toggleTheme } = useContext(ThemeContext);

  return (
    <article className={theme}>
      <h3>{heading}</h3>
      <p>{summary}</p>
      <button onClick={toggleTheme}>Toggle</button>
    </article>
  );
};
```

Then, whenever we click the **Toggle** button, it will change the theme and trigger a re-render:

Figure 2.4: Using a theme context

In React, the Context API allows you to create and manage global state that can be accessed by components throughout your application. This capability enables you to combine multiple context providers, each representing a different slice or aspect of your application's state.

By using separate context providers, such as one for security, another for logging, and potentially others, you can organize and share related data and functionality efficiently. Each context provider encapsulates a specific concern, making it easier to manage and update related state without impacting other parts of the application:

```
import InteractionContext from "xui/interaction-context";
import SecurityContext from "xui/security-context";
import LoggingContext from "xui/logging-context";

const Application = ({ children }) => {
```

```
  const context = {}; // ... define values for context

  //... define securityContext
  //... define loggingContext

  return (
    <InteractionContext.Provider value={context}>
      <SecurityContext.Provider value={securityContext}>
        <LoggingContext.Provider value={loggingContext}>
          {children}
        </LoggingContext.Provider>
      </SecurityContext.Provider>
    </InteractionContext.Provider>
  );
};
```

This example demonstrates the usage of React's Context API to create and combine multiple context providers within an application component. `InteractionContext.Provider`, `SecurityContext.Provider`, and `LoggingContext.Provider` are used to wrap the children components and provide the respective context values.

There are several additional built-in Hooks available in React that are used less frequently. In the following chapters, we will introduce these Hooks as needed, focusing on the ones that are most relevant to the topics at hand.

Summary

In this introductory chapter, we covered essential concepts of React and laid the groundwork for your React development journey. We explored the concept of thinking in components, emphasizing the importance of breaking down applications into reusable and modular pieces. By adopting this mindset, you'll be able to create maintainable and scalable code bases.

Furthermore, we introduced you to the most commonly used Hooks in React, such as `useState` and `useEffect`, which empower you to manage state and handle side effects efficiently within functional components. These Hooks provide the flexibility and power to build dynamic and interactive UIs.

By mastering the fundamental principles of React and familiarizing yourself with the concept of thinking in components, you are now well prepared to dive deeper into the world of React development. In the upcoming chapters, we will explore more advanced topics and tackle real-world challenges, enabling you to become a proficient React developer.

Remember – React is a powerful tool that opens endless possibilities for creating modern and robust web applications. In the upcoming chapter, we will delve into the process of breaking down a design into smaller components and explore effective strategies for organizing these components. We will emphasize the importance of reusability and flexibility, ensuring that our components are adaptable to future changes.